Furthermore, before sending your request, you have the option to utilize the **`withoutDeprecationHandling`** function, which ensures that your application does not rely on any functionalities that the PHP language or its associated libraries have deprecated. Disabling deprecation handling will result in the transformation of deprecation warnings into exceptions, causing your test to fail.

```
$response = $this->withoutDeprecationHandling()->get('/');
```

To assert that code inside a given closure throws an exception of the designated type, use the **`assertThrows`** method:

```
$this->assertThrows(
  fn () => (new ProcessOrder)->execute(),
  OrderInvalid::class
);
```

File Upload Testing

For the purpose of testing, you can employ the **`fake`** function from the **`Illuminate\Http\UploadedFile`** class to generate mock files or images. This simplifies the testing of file uploads, particularly when used alongside the **`Storage`** facade's **`fake`** method. For instance, you can swiftly test an avatar upload form by combining these two functionalities:

```
<?php
namespace Tests\Feature;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Tests\TestCase;
class ExampleTest extends TestCase
{
  public function testUpload(): void
  {
    Storage::fake('cat');
    $file = UploadedFile::fake()->image('cat.jpg');
    $response = $this->post('/cat', [
      'cat' => $file,
    ]);
    Storage::disk('cats')->assertExists($file->hashName());
  }
```